REMS

Real Estate Management System

Version:	1.0.0
Package:	com.nativecodex.rems
Platform:	Flutter + Firebase
Author:	NativeCodex
Date:	2026

Table of Contents

1. Product Overview

REMS (Real Estate Management System) is a full-stack Flutter + Firebase mobile application that serves as a complete property operating system. It connects tenants, caretakers, property managers, landlords/owners, and platform administrators in a single unified experience.

Core Concept

Think of the app as an operating loop: a tenant finds a unit requests access the caretaker/owner reviews access is approved the lease starts and rent and maintenance become ongoing daily operations. Firebase keeps everything in sync.

Working Name

REMS - Real Estate Management System

Package Identifier

```
com.nativecodex.rems
```

Key Capabilities

- Property discovery with search and filters

2. Architecture & Tech Stack

Frontend

Built with Flutter 3.41.x using Riverpod for state management. The app follows a clean architecture pattern with core, data, and feature layers.

Backend

Firebase serves as the full backend stack: Firestore for data, Authentication for user management, Storage for media, Cloud Messaging for push notifications, Cloud Functions for backend logic, Remote Config for feature flags, and Analytics + Crashlytics for monitoring.

Technology Stack

Layer	Technology	Purpose
UI	Flutter 3.41	Cross-platform mobile UI
State	Riverpod 2.6	State management
Auth	Firebase Auth	Email/Google/Phone auth
DB	Cloud Firestore	Real-time document DB
Storage	Firebase Storage	Images, docs, receipts
Push	FCM + Local	Notifications
Config	Remote Config	Feature flags, gating
Analytics	Firebase Analytics	Usage tracking
Crash	Crashlytics	Error reporting

State Management: Riverpod

Riverpod was chosen for its compile-time safety, testability, and clean separation of concerns. Providers are used for auth state, user data, repositories, and feature state. The app uses StreamProvider for real-time Firestore listeners, FutureProvider for one-time fetches, and ChangeNotifierProvider for auth flow state.

3. User Roles & Permissions

REMS defines five distinct user roles, each with unique permissions, UI, and data scope. Access is enforced through Firestore security rules and client-side role-based routing.

Role	Scope	Primary Actions
Tenant	Own profile & unit	Browse, request, pay, maintain
Caretaker	Assigned properties	Approve, manage, maintain
Owner	Owned properties	Monitor, report, manage staff
Manager	Managed properties	Operations oversight
Admin	Entire platform	Users, properties, plans, audit

Security Principle

No client-side role elevation. Payment status, approval status, lease creation, and role changes are server-controlled through Cloud Functions. Firestore rules enforce property-scoped data isolation.

Role-Based Navigation

Each role sees a different bottom navigation bar tailored to their workflow:

- Tenant: Home | Properties | Requests | Messages | Profile

4. Firestore Data Model

Cloud Firestore is the primary database, organized into 14 collections. Every collection is property-scoped, ensuring data isolation between properties and roles. Below is the complete schema.

Collection	Description
users	User profiles with role, status, subscription tier, property/unit links
properties	Property metadata, owner/manager links, unit counts
buildings	Building info within a property
units	Individual units with rent, deposit, occupancy, status
access_requests	Tenant access requests with approval workflow
leases	Active and historical lease documents
payments	Payment records with method, reference, receipt
maintenance_tickets	Maintenance requests with priority, status, assignment
notifications	Push and in-app notifications per recipient
announcements	Property-wide announcements targeting audiences
staff	Property staff assignments and permissions
audit_logs	System-wide audit trail for compliance
subscriptions	Owner subscription plans and billing
messages	Direct messages between users within a property

Key Data Relationships

Properties own Buildings -> Buildings own Units. Users are linked via role-specific fields: ownerId, managerId, caretakerId, tenantId. Access requests bridge tenants to units through an approval workflow. Leases are created upon approval. Payments reference tenant, property, and unit.

Users Collection Schema

```
users/{uid}
  fullName, phone, email, role, status
  photoUrl, county, idNumber
  companyName, businessRegistration (owner)
  employmentStatus, occupation (tenant)
  emergencyContact, nextOfKin (tenant)
  assignedPropertyCode (caretaker)
  currentPropertyId, currentUnitId
  subscriptionTier (free/premium)
  isVerified, preferredLanguage
  createdAt, lastLoginAt
```

Units Collection Schema

```
units/{unitId}
  propertyId, buildingId
  unitNumber, unitType, bedrooms
  rentAmount, depositAmount
  occupied, tenantId, caretakerId
  status (vacant/occupied/maintenance)
  photos[], createdAt
```

5. Authentication & Security

Authentication Methods

Firebase Authentication provides three sign-in methods, all configured in the Firebase Console:

- Email/Password with email verification

Account Status Flow

Every new account starts with status = "pending". The user sees a limited waiting screen until an admin or owner approves them. Possible statuses:

Status	Meaning
pending	Awaiting approval, limited access
active	Full access based on role
rejected	Application denied
suspended	Temporarily disabled

Firestore Security Rules

Security rules enforce strict data isolation by role and ownership scope:

- Tenants can ONLY read/write their own profile and requests

6. Feature Map by Role

Tenant Features

Feature	Free	Premium
Property Discovery	YES	YES
Access Requests	YES	YES
Rent Reminders	YES	YES
Basic Maintenance	YES	YES
Payment History	YES	YES
Basic Dashboard	YES	YES
Ads	Shown	None
Multi-Property	1	Unlimited
Advanced Analytics	-	YES
Lease Automation	-	YES
PDF Receipts	-	YES
Smart Notifications	-	YES
AI Tools	-	Phase 3

Caretaker Features

Feature	Free	Premium
Pending Approvals	YES	YES
Unit Management	YES	YES
Maintenance Queue	YES	YES
Tenant List	YES	YES
Basic Reports	YES	YES
Task Assignment	-	YES
Bulk Actions	-	YES
Advanced Reports	-	YES

Owner Features

Feature	Free	Premium
Property Overview	1 property	Unlimited
Revenue Dashboard	Basic	Advanced
Occupancy Tracking	YES	YES
Staff Management	-	YES
Financial Reports	-	YES
PDF Export	-	YES
Cloud Backups	-	YES
AI Predictions	-	Phase 3

7. Navigation & UI Structure

The app uses a shell-based navigation pattern where each role has its own bottom navigation bar and set of screens. Authentication state determines which shell is loaded. Routing is handled via named routes with onGenerateRoute.

Screen Flow Diagram

```
Splash -> Welcome -> Role Selection -> Register/Login -> Verification
                                                          |
                          +-----------+----------+------+------+
                          |           |          |             |
                      Tenant    Caretaker    Owner         Admin
                          |           |          |             |
                    Home      Dashboard   Overview      Users
                    Properties  Requests   Properties    Properties
                    Requests    Units      Finance       Plans
                    Messages    Tasks      Reports       Analytics
                    Profile     Profile    Profile       Audit
```

UI Design Principles

- Clean, modern Material 3 design with Inter font

8. Onboarding Flow

The onboarding process takes a user from first launch to an active account in a series of well-defined steps.

1. Splash Screen

App logo with Firebase initialization. Auto-routes to dashboard if session exists, else to Welcome screen.

2. Welcome Screen

Login, Register, and Guest Viewer options. Guest mode only shows public property listings.

3. Role Selection

User picks their role: Tenant, Caretaker, Owner/Landlord, or Property Manager. Role determines form fields and UI.

4. Registration Form

Role-specific fields. Common: name, phone, email, password, county. Tenant adds: ID, next-of-kin, employment. Owner adds: company, business reg. Caretaker adds: property code.

5. Verification

Email verification link sent. Phone OTP optional. Account status = "pending" until approved by admin/owner.

6. Waiting Screen

User sees a pending-approval screen until their status changes to "active".

9. Tenant Journey

The tenant experience is designed around discovering a property, requesting access, and then managing their rental life through the app.

A. Discovering Properties

The Properties screen shows available units with search bar and filter chips (rent range, unit type, bedrooms, furnishing). Each property card shows the building name, location, available units, rent amount, and an image.

B. Requesting Access

Tapping a property shows unit details: rent, deposit, type, features. The "Request Access" button creates a Firestore access_requests document with status = "pending". The tenant enters a desired move-in date.

C. Approval Flow

The request is routed to the assigned caretaker and/or owner. They can approve, reject, or request more information. On approval, the system creates a lease, updates unit occupancy, links the tenant, and sends a notification.

D. Daily Usage

The tenant dashboard shows: current unit info, rent due date and balance, announcements, maintenance shortcuts, recent receipts, and an ad tile (free tier). The tenant can pay rent, submit maintenance tickets, message caretaker, view lease documents, and download receipts.

10. Caretaker Journey

Caretakers are the operational backbone of the app. They manage day-to-day property operations and tenant interactions.

Dashboard Layout

The caretaker dashboard shows a property selector at the top, followed by summary cards with counts for pending approvals, unpaid rent, maintenance tickets, and vacant units. Below are sections for quick access to requests and maintenance.

Daily Workflow

- Check and process pending access requests (approve/reject)

Approval Decision Flow

```
Tenant applies -> Caretaker opens request
  -> Reviews tenant data & documents
  -> Checks unit availability
  -> Confirms terms
  -> Approves or Rejects
  -> System auto-creates lease & updates unit
  -> Notification sent to tenant
```

11. Owner / Landlord Journey

Owners get a macro-level view of their property portfolio with financial oversight and strategic decision-making tools.

Dashboard Layout

The owner overview screen displays KPI cards: monthly income, occupancy rate, arrears, and active properties. A revenue chart shows trends over time. Quick action buttons let owners add properties or generate reports.

Owner Capabilities

- View multiple properties in a single portfolio view

Premium Upsell

Free-tier owners are limited to 1 property and basic reports. Premium unlocks unlimited properties, advanced analytics, PDF export, staff management, and AI tools (Phase 3). A premium banner appears at strategic points.

12. Admin Console

The admin panel provides full platform oversight. Admins manage users, properties, subscription plans, ad placement rules, and system analytics.

Admin Sections

Users

View all users by role and status. Approve/reject pending accounts. Search and filter. Suspend or activate accounts.

Properties

View all properties across the platform. Edit property details. Manage property status.

Plans

Configure subscription tiers: Free and Premium. Set pricing, features, and limits per plan.

Analytics

Platform-wide metrics: total users, active properties, revenue, tenant counts. Charts for growth tracking.

Audit

System audit logs with actor, action, target, and timestamp. Export for compliance.

13. Monetization: Free & Premium

REMS uses a freemium model with ad-supported free tier and subscription-based premium. This ensures the app is accessible while generating revenue.

Free Tier

- Ad-supported (ads in home feed, property list, reports preview)

Premium Tier

- No ads anywhere in the app

Ad Placement Rules

Ads are carefully placed to avoid breaking trust:

- NO ads on payment confirmation screens

14. Folder Structure

The Flutter codebase follows a feature-first architecture with clean separation of concerns. Below is the complete directory tree.

```
lib/
  main.dart                    # App entry point
  core/
    theme/                     # AppTheme, AppColors
    constants/                 # AppConstants, FirestoreConstants
    utils/                     # Helpers, Validators
    routes/                    # AppRoutes
  data/
    models/                    # 13 domain models
    repositories/              # 7 data repositories
    services/                  # FirebaseService, AuthService
  features/
    auth/        screens/      # Splash, Welcome, Login, Register,
                               #   RoleSelection, Verification
                 providers/    # AuthNotifier
    tenant/      screens/      # Home, Properties, Requests,
                               #   Messages, Profile, Shell
    caretaker/   screens/      # Dashboard, Requests, Units,
                               #   Tasks, Profile, Shell
    owner/       screens/      # Overview, Properties, Finance,
                               #   Reports, Profile, Shell
    admin/       screens/      # Users, Properties, Plans,
                               #   Analytics, Audit, Shell
    properties/  screens/      # UnitDetail
  widgets/                     # PropertyCard, UnitCard, RequestCard,
                               #   MaintenanceCard, AdBanner, etc.
android/
  app/
    src/main/
      kotlin/com/nativecodex/rems/   # MainActivity.kt
      AndroidManifest.xml
      google-services.json           # Firebase config
```

15. Firebase Services

REMS uses the following Firebase services, all integrated through the FlutterFire plugins:

Service	Usage
Authentication	Email/password, Google, Phone OTP
Cloud Firestore	All app data (14 collections)
Firebase Storage	Profile photos, property images, lease PDFs, receipts
Cloud Messaging	Push notifications for approvals, payments, maintenance
Cloud Functions	Backend automation: approvals, payments, reminders
Remote Config	Feature flags, premium gating, ad rules
Analytics	Usage tracking, funnel analysis
Crashlytics	Crash reporting and stability monitoring
App Check	Protect Firestore/Functions from abuse

Firestore Indexes Required

The following composite indexes must be created in the Firebase Console:

```
propertyId + status
tenantId + createdAt
propertyId + createdAt
unitId + occupied
ownerId + status
caretakerId + status
propertyId + priority + status
recipientId + read + createdAt
propertyId + unitType + rentAmount
```

16. Cloud Functions Logic

Cloud Functions provide server-side automation for critical workflows. These ensure data integrity and prevent client-side manipulation of sensitive operations.

onRequestCreated

Create notification for caretaker, notify owner, set SLA timer

onRequestApproved

Create lease, update unit to occupied, link tenant, send approval notification

onPaymentRecorded

Generate receipt PDF, update balance, notify tenant, update ledger

onMaintenanceCreated

Notify caretaker, assign priority based on category, escalate if unresolved

onDueDateApproaching

Send rent reminders, flag account for late fee, apply penalty rules if premium

> **Important**
>
> Do NOT let the client directly decide: payment status, approval status, lease creation, or role elevation. All such operations MUST go through Cloud Functions or server-side security rules.

17. Phase Roadmap

MVP 1 - Core (Current)

Feature	Status
User registration & auth	Implemented
Role selection & profile	Implemented
Property/unit browsing	Implemented
Access requests & approvals	Implemented
Role-based dashboards	Implemented
Push notifications	Framework ready
Firestore data model	Implemented
Storage for images/docs	Service ready
Ad placements (free tier)	Widget ready
Premium gating framework	Widget ready

MVP 2 - Operations (Next)

Feature	Priority
Rent payments & receipts	High
Lease document handling	High
Visitor management	Medium
Announcement system	Medium
Meters & utilities	Medium
Move-in / move-out flow	High
Late payment penalties	Medium
Multi-property dashboards	High
PDF export	Medium
Advanced search & filtering	Medium
Enhanced analytics	Medium

MVP 3 - Advanced Platform

Feature	Priority
AI assistant	Low
Anomaly detection	Low
Predictive rent collection	Low
Vacancy forecasting	Low
Maintenance prediction	Low
Advanced audit logs	Medium
Custom branding (agencies)	Low
SMS/email automation	Medium
Offline sync improvements	Medium
Web admin portal	Medium
IoT / smart meter integration	Low